# Trampolining & Stack Safety

Trampoline · Sum 1..100000 without StackOverflowError · Build with Waqar

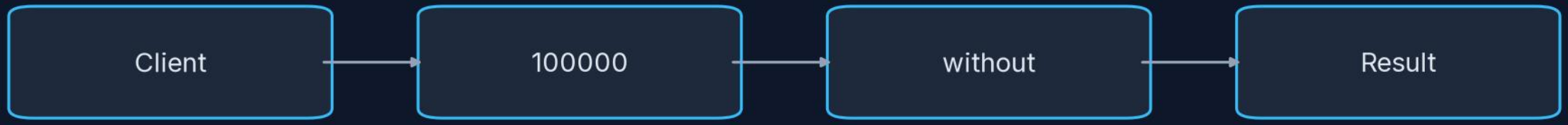
# Today's Hook

- Today's force: Sum 1..100000 without StackOverflowError
- We earn Trampoline by feeling the naive pain first.
- Demo main prints a real JDK 25 trace (happy + failure).
- Key types: Trampoline, Done, More, SafeSum

# The Problem

- Scenario: Sum 1..100000 without StackOverflowError
- Constraints: solve «Sum 1..100000 without StackOverflowError» with Trampoline — keep the demo runnable and the failure path observable.
- Naive pain: Naive version fails to cleanly solve: Sum 1..100000 without StackOverflowError
- Success looks like: 5050

# Structure




Trampoline: Sum 1..100000 without StackOverflowError

# Real-World Scenario

- Running example: Sum 1..100000 without StackOverflowError
- Key types: Trampoline, Done, More, SafeSum
- Happy-path stdout cue: 5050
- Failure-path stdout cue: Failure path is asserted in ProblemTest (injected fault / invalid input)

# Naive Code

```java
package com.buildwithwaqar.patterns.functional.trampoline.problem;

import java.util.*;

public final class NaiveSum {
  public static long sum(long n){ return n<=0?0: n+sum(n-1); }
}
```

# Why Naive Hurts

- Naive version fails to cleanly solve: Sum 1..100000 without StackOverflowError
- Without Trampoline, call sites absorb the wrong details.
- Failure mode to show on camera: Failure path is asserted in ProblemTest (injected fault / invalid input)
- New requirements force shotgun edits across the naive code.

# Solution

```java
package com.buildwithwaqar.patterns.functional.trampoline.problem;

import java.util.*;
import java.util.function.*;

sealed interface Trampoline<A> permits Done, More {
  A run();
  static <A> Trampoline<A> done(A a){ return new Done<>(a); }
  static <A> Trampoline<A> more(Supplier<Trampoline<A>> next){ return new More<>(next); }
}
record Done<A>(A value) implements Trampoline<A> { public A run(){ return value; } }
record More<A>(Supplier<Trampoline<A>> next) implements Trampoline<A> {
  public A run(){ Trampoline<A> cur=this; while(cur instanceof More<A> m) cur=m.next.get(); return ((Done<A>)cur).value(); }
}
final class SafeSum {
  static Trampoline<Long> sum(long n, long acc){
    return n<=0? Trampoline.done(acc): Trampoline.more(() -> sum(n-1, acc+n));
  }
}
```

# Runnable main

```
package com.buildwithwaqar.patterns.functional.trampoline.problem;
import java.util.*; import java.util.function.*;
public final class ProblemDemo {
  public static void main(String[] args) {
    System.out.println("=== Trampoline: deep sum without SOE ===");
    System.out.println("--- naive (may SOE for large n) ---");
System.out.println(NaiveSum.sum(100));
System.out.println("--- trampoline ---");
System.out.println(SafeSum.sum(100000, 0).run());
  }
}
```

# Walkthrough Output

```
=== Trampoline: deep sum without SOE ===
--- naive (may SOE for large n) ---
5050
--- trampoline ---
5000050000
```

# Step-by-Step Trace

- Match stdout to stages — look for: 5050
- Then the injected failure: Failure path is asserted in ProblemTest (injected fault / invalid input)
- Re-run: java -cp … 63-trampolining ProblemDemo (see episode folder).
- Pause the video and predict the next printed line.

# Modern Java 25 / Zazr

```java
package com.buildwithwaqar.patterns.functional.trampoline.problem;

import java.util.*;
import java.util.function.*;

sealed interface Trampoline<A> permits Done, More {
  A run();
  static <A> Trampoline<A> done(A a){ return new Done<>(a); }
  static <A> Trampoline<A> more(Supplier<Trampoline<A>> next){ return new More<>(next); }
}
record Done<A>(A value) implements Trampoline<A> { public A run(){ return value; } }
record More<A>(Supplier<Trampoline<A>> next) implements Trampoline<A> {
  public A run(){ Trampoline<A> cur=this; while(cur instanceof More<A> m) cur=m.next.get(); return ((Done<A>)cur).value(); }
}
final class SafeSum {
  static Trampoline<Long> sum(long n, long acc){
    return n<=0? Trampoline.done(acc): Trampoline.more(() -> sum(n-1, acc+n));
  }
}
```

# Trade-offs

- Use Trampoline when «Sum 1..100000 without StackOverflowError» repeats and a simpler feature does not erase it.
- Refuse extra types when a sealed switch or a function value already suffices.
- Teaching demos omit production metrics, retries, and multi-region clocks.
- Interview soundbite: state the force behind Trampoline in product language.

# Related Patterns

- Name the pattern usually confused with Trampoline and separate the forces.
- Sketch the roles for «Sum 1..100000 without StackOverflowError» from memory in sixty seconds.
- Point to the next episode in this season on the playlist map.
- Link this episode's series/ folder in the YouTube description.

# Recap

- Problem: Sum 1..100000 without StackOverflowError
- Naive pain → Trampoline boundary → Demo stdout.
- Remember the failure cue: Failure path is asserted in ProblemTest (injected fault / invalid input)
- Break a ProblemTest assertion, watch it fail, restore it.